

EU AI Act Compliance Copilot

A multi-agent RAG system from first commit to production: FastAPI, LangGraph, pgvector, Redis, MLflow, Docker, GitHub Actions, Prometheus, Langfuse.

How to use this blueprint. Read sections 1–3 once to understand the system. Then work through the phases in section 4 with the starter repo (ai-act-copilot.zip): each phase has functions to write, tests that tell you when you're done, and study-notes days to read alongside.

Section 5 lists the numbers to measure; section 6 turns them into resume bullets and an interview pitch. Decision support, not legal advice: the product is designed around that fact.

Build

RAG · agents · multi-agent LangGraph · FastAPI

LLMOps

prompts · fallback · cache · evals · tracing

MLOps

MLflow tracking · registry gate · drift monitoring

DevOps

Docker · CI/CD · EU deployment ·
Prometheus/Grafana

Contents

Blueprint

- | | | | |
|---|-----------------------------|---|---|
| 1 | The problem and the product | 5 | Results to measure (fill these in – they are your resume numbers) |
| 2 | Architecture | 6 | Resume material |
| 3 | How each part works | 7 | Stretch goals |
| 4 | The phases | | |

Blueprint

The complete design, build plan and resume material for the flagship project.

A production-grade, end-to-end AI system for your portfolio: RAG over the EU AI Act, a multi-agent pipeline that classifies an AI system's risk and maps its obligations, a classic ML model with a full MLOps lifecycle, and the LLMOps and DevOps practices that make it run reliably in production.

Decision support, not legal advice. The product is designed around that fact: it cites the law, flags uncertainty, verifies its own reports and asks a human when information is missing.

1 • The problem and the product

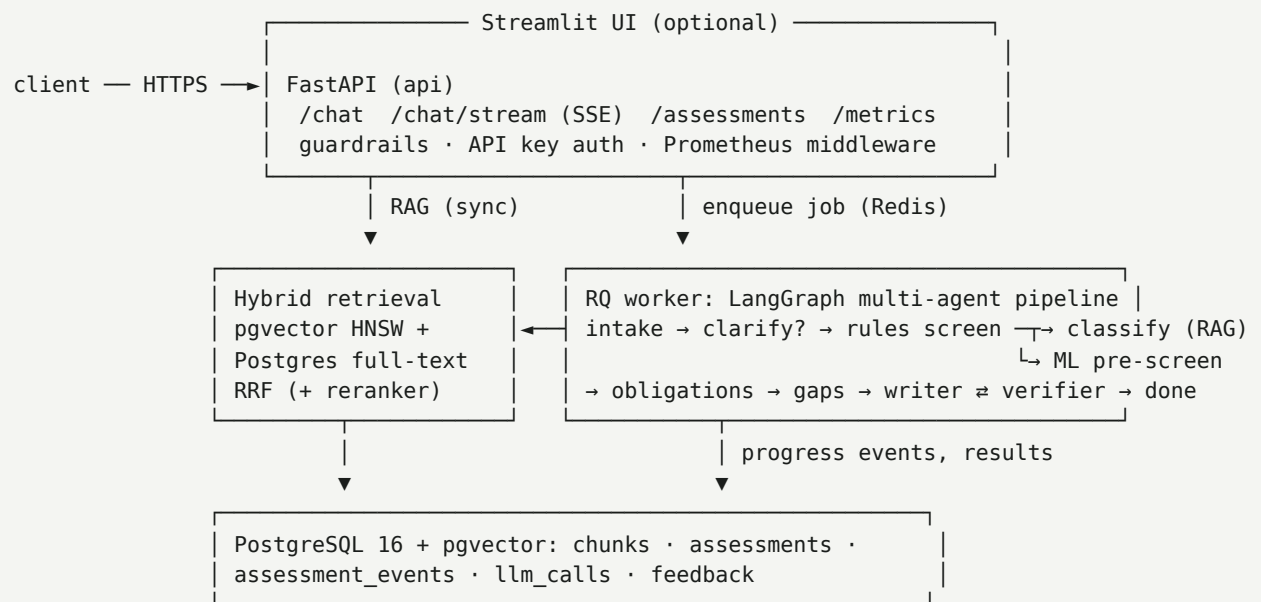
Problem. The EU AI Act applies to every company that builds or uses AI in the EU. Product teams must answer questions like "Is our CV-screening feature high-risk?", "What do we have to do, and by when?" and "Do we need to tell users they're talking to a bot?". Lawyers are expensive and slow; the regulation is 140+ pages plus annexes, and it was amended by the 2026 AI Omnibus (high-risk deadlines moved to 2 December 2027 for Annex III systems and 2 August 2028 for Annex I systems).

Product. Two features, one API:

1. **Ask the AI Act** (`POST /chat`): questions answered only from the regulation's text, with numbered citations that are verified in code, and an honest "I couldn't find this" when the text doesn't say.
2. **Assess an AI system** (`POST /assessments`): describe a system in plain language; a team of agents extracts a structured profile, asks clarifying questions if needed, screens it with deterministic rules and an ML model, classifies it with citations, looks up the obligations and deadlines, analyses gaps against what the team already does, writes a report, and verifies that report before returning it.

Why it stands out in a portfolio. It is not "chat with PDF". It combines retrieval, structured agents, deterministic rules, a trained model, human-in-the-loop, evaluation, observability and deployment, in a domain German and EU employers care about right now.

2 • Architecture



LLM providers: Ollama (local, free) | Claude / OpenAI (prod) – one interface, fallback chain
MLOps: MLflow tracking + registry ("champion" alias) · drift check (weekly GitHub Action)
Observability: Prometheus + Grafana · Langfuse traces · llm_calls table · structured JSON logs
CI/CD: GitHub Actions → lint, unit, integration (service containers), eval gate → GHCR →
deploy (EU) → smoke test

COMPONENTS

Component	Technology	Why this choice
API	FastAPI, Pydantic, sse-starlette	Typed request/response models, automatic OpenAPI docs, async streaming
Background jobs	Redis + RQ	Multi-minute agent runs never block HTTP requests; simple and well understood
Orchestration	LangGraph	Explicit graph, parallel branches, conditional routing, loops; widely asked for in job ads
Vector + keyword search	PostgreSQL 16, pgvector (HNSW), tsvector (GIN)	One database for vectors, text search, jobs and logs; transactions; SQL filters
Embeddings	nomic-embed-text via Ollama (768-d)	Free, local, runs on CPU in production too
LLMs	Ollama (dev), Claude / OpenAI (prod)	Provider interface + fallback; develop free, deploy reliable
Classic ML	scikit-learn TF-IDF + logistic regression	Fast, cheap, explainable second opinion; real MLOps lifecycle
Experiment tracking	MLflow (tracking server + registry)	Industry standard; aliases for champion/challenger
Tracing	Langfuse + <code>llm_calls</code> table	Per-call tokens, latency, cost, prompt version
Metrics	Prometheus + Grafana	Golden signals + LLM tokens and cost
Packaging	Docker multi-stage, Docker Compose (9 services)	Same image everywhere, non-root, health checks
CI/CD	GitHub Actions, GHCR	Lint, tests, integration tests, eval gate, image build, deploy, smoke test
Load testing	Locust	Find limits before users do

DATA MODEL (MIGRATIONS/001_INIT.SQL)

- `chunks`: id, provision (article/annex), title, chapter, text, content hash, `embedding vector(768)`, generated `tsvector`; HNSW and GIN indexes.
- `assessments` + `assessment_events`: job status, input, result (JSONB), progress stream.
- `llm_calls`: trace id, agent, provider, model, prompt name/version, tokens, latency, cost.
- `feedback`: thumbs up/down on answers and assessments.

3 • How each part works

3.1 INGESTION (PHASE 1)

1. Fetch the regulation from EUR-Lex (or read a saved HTML copy), convert to clean lines.

2. **Parse by legal structure:** chapters, articles and annexes become `Provision` objects; recitals are skipped (they mention article numbers in running text).
3. **Legal-aware chunking:** split at numbered paragraphs, pack up to ~1,800 characters, split overlong paragraphs at sentence ends, prepend a context header ("EU AI Act · Article 5 · Prohibited AI practices · CHAPTER II").
4. **Idempotent upsert:** content hashes mean re-ingesting only re-embeds changed chunks; removed provisions are deleted. When the consolidated text with the Omnibus amendments is published, point the ingester at it and re-run.

3.2 RETRIEVAL (PHASE 2)

- Vector search (pgvector, cosine, HNSW) catches meaning ("can I screen job applicants with AI?").
- Keyword search (Postgres full-text, `websearch_to_tsquery`) catches exact terms ("Annex III", "CE marking").
- **Reciprocal Rank Fusion** merges the two rankings without score calibration.
- Optional **cross-encoder reranker** on the top 30.
- Measured with recall@5 and MRR on 25 labelled questions (`evals/retrieval_eval.jsonl`), per method.

3.3 GROUNDED ANSWERS (PHASE 3)

Numbered documents in, `[n]` citations out, checked in code: citing a document that doesn't exist marks the answer as unverified. Input guardrails reject empty, oversized and obvious injection attempts; PII (emails, IBANs, phone numbers) is redacted before any model or log sees it. Streaming via server-sent events.

3.4 THE AGENTS (PHASES 4–5)

Agent	Type	Job	Safety net
Intake	LLM, structured output	Description → <code>SystemProfile</code> (15 typed fields) + missing-info questions	Pydantic validation + retries; description treated as data
Clarify	Human-in-the-loop	Returns questions; a new run starts when answered	No guessing on key facts like provider vs deployer
Rules screen	Deterministic code	Flags: Annex III area, prohibited patterns, transparency, GPAL, safety components	Same input → same flags, always
ML pre-screen	Trained classifier	Predicts the Annex III area in milliseconds	Disagreement with the LLM becomes a review flag
Classifier	LLM + RAG workflow	Plans searches → retrieves → classifies with citations	Unsupported citations dropped; rule disagreement lowers confidence
Obligations	Deterministic lookup	Category + role → obligations and post-Omnibus deadlines	Curated table, auditable, not generated
Gaps	Deterministic	Obligations vs current practices → prioritised gaps	Priority from deadlines and status
Writer	LLM	Markdown report from structured results only	Told not to add facts
Verifier	Deterministic	Checks citations exist, classification stated, every deadline present, disclaimer	Failed → back to the writer (max 3 attempts)

Patterns used (notes Day 67): routing (clarify vs continue), parallelisation (classify || ML pre-screen), prompt chaining, evaluator-optimiser (writer ⇌ verifier), human-in-the-loop. The design principle: *LLMs read and write; code decides what must be exact.*

3.5 MLOPS (PHASE 6)

```

seed data → LLM-generated drafts → human review → versioned CSV (git/DVC)
    ↳ train (TF-IDF + LR) → evaluate (macro F1, per class) → MLflow run
        ↳ register version → promotion gate (min F1, beats champion, no class regression)
            ↳ alias "champion" → served in the pipeline (with local fallback)
production inputs → weekly PSI drift check (GitHub Action) → issue → relabel + retrain

```

3.6 LLMOPS (PHASE 7)

- **Prompts as versioned files** (prompts/*.yaml); every call logs name + version.
- **Provider abstraction + fallback chain** (LLM_PROVIDERS=anthropic,openai,ollama); streaming only falls back before the first token.
- **Semantic cache** for /chat with a strict threshold, TTL and size limit; hit rate tracked.
- **Evals**: 25 retrieval questions; 20 labelled assessment scenarios scored on category, Annex III area, transparency and citation recall; LLM-judge agreement (TPR/TNR) utilities.

- **CI eval gate:** PRs touching prompts/agents/retrieval run the scenario evals and fail below 80% pass rate; the report is posted as a PR comment.
- **Tracing:** every call's tokens, latency and cost go to Postgres and Langfuse; Prometheus counters feed the cost dashboard.

3.7 DEVOPS AND PRODUCTION (PHASE 8)

- **Docker:** multi-stage build, non-root user, health check, `--proxy-headers`.
- **Compose:** api, worker, db (pgvector), redis, ollama, mlflow, prometheus, grafana, ui.
- **Environments:** dev (Compose, Ollama), test (in-memory fakes), prod (managed services). Config only through environment variables; `API_KEY` mandatory in prod.
- **CI (ci.yml):** ruff lint + format check → unit tests → integration tests against Postgres/pgvector and Redis service containers → eval gate → Docker build with layer cache.
- **CD (cd.yml):** after CI on `main`: build and push to GHCR (tagged with the commit SHA), deploy, smoke-test `/health`. Rollback = redeploy the previous SHA.
- **Monitoring:** Prometheus scrapes `/metrics` (request rate, errors, p95 latency by route template, LLM tokens, cost, LLM latency); Grafana dashboard provisioned from the repo; `/ready` checks the database and Redis.
- **Load testing:** Locust scenario mixing chat, assessments and health checks.

Default deployment: Render, Frankfurt region (simple, EU data residency):

Render resource	Runs
Web Service (Docker image from GHCR)	the API
Background Worker (same image, command <code>rq worker assessments</code>)	the agent pipeline
Private Service (image <code>ollama/ollama</code> , CPU)	embeddings only (<code>omic-embed-text</code> is small enough for CPU)
PostgreSQL (enable the <code>vector</code> extension)	data
Key Value (Redis-compatible)	job queue

Chat and agent LLM calls use a cloud API in production (`LLM_PROVIDERS=anthropic,openai`), since cloud CPUs are too slow for 8B chat models.

Alternatives (only the deploy step changes): Google Cloud Run (europe-west3) + Cloud SQL + Memorystore; Azure Container Apps (Germany West Central) + Azure Database for PostgreSQL (pgvector supported) + Azure Cache for Redis; Fly.io (Frankfurt). Mentioning why you chose one over the others is a good interview answer.

3.8 SECURITY AND RESPONSIBLE AI

- OWASP LLM Top 10 threat model in `docs/RUNBOOK.md` (injection, sensitive data, output handling, excessive agency, unbounded consumption).
- The agents have **no write tools**: the worst an injection can do is produce a wrong report, which the verifier and the human reviewer check.

- PII redaction before models and logs; API key auth; input size limits; rate limiting at the platform level.
 - **Dogfooding:** `docs/AI_ACT_SELF_ASSESSMENT.md` classifies this tool itself under the AI Act (it interacts with people and generates text → Article 50 transparency; it doesn't make decisions about natural persons → not high-risk), and `docs/MODEL_CARD.md` documents the classifier.
-

4 • The phases

Each phase has code to write (functions marked `# YOUR CODE`, with precise docstrings), tests that tell you when you're done (`make test-phase P=n`), and study-notes days to read alongside. At 2–3 hours a day, the whole project takes about 6–7 weeks.

Phase	Build	Files	Tests	Notes	Time
0	Run the stack, pull models, migrate, read the code	—	test_phase0 (passes already)	Days 9, 86	1–2 days
1	Parse the Act, legal-aware chunking, ingest	ingest/parse.py , ingest/chunk.py	9	Days 53, 56	2–3 days
2	RRF, hybrid search, recall@k, MRR; compare methods	retrieval/search.py	6	Days 51, 52, 57, 58	2–3 days
3	Grounded answers, citation checks, guardrails, POST /chat	rag/answer.py , rag/guardrails.py , api/routes_chat.py	10	Days 44, 59, 64, 76, 88	3 days
4	Rules screen, intake agent, classifier agent	agents/rules.py , agents/intake.py , agents/classifier.py	9	Days 45, 67–69, S7	4 days
5	Obligations, gaps, verifier, LangGraph wiring, jobs, UI	agents/obligations.py , gaps.py , verifier.py , graph.py	8	Days 73–75, 67	5 days
6	Training data, classifier, MLflow, promotion gate, drift	mlops/train.py , registry.py , drift.py	5	Days 13–17, 29, 85	4 days
7	Fallback, semantic cache, eval scoring, run evals, Langfuse	llmops/fallback.py , cache.py , evals.py	5	Days 47, 49, 50, 87, 89	4 days
8	Prometheus metrics, Grafana, CI/CD, deploy, load test	observability/metrics.py , workflows	2 + CI	Days 85–90	4–5 days
9	README results, model card, self-assessment, runbook, demo	docs/	—	Days 91–96	3 days

DEFINITION OF DONE FOR EACH PHASE

- **Phase 1:** make ingest stores all articles and annexes; spot-check 5 chunks in the database (SELECT id, left(text, 200) FROM chunks LIMIT 5).
- **Phase 2:** make eval-retrieval prints a table for vector, keyword and hybrid (and reranked); hybrid is at least as good as the best single method. Put the table in the README.
- **Phase 3:** /docs → POST /chat answers with valid citations; an injection attempt returns 400; /chat/stream streams.
- **Phase 4:** run intake + classifier on 5 scenarios by hand; categories and citations look right.

- **Phase 5:** a full assessment through the UI or API: progress events stream, the report passes verification, and missing info triggers questions.
- **Phase 6:** at least 300 reviewed training rows; MLflow shows runs; the champion alias is set; `make drift` works.
- **Phase 7:** `make evals` pass rate recorded for two models (e.g. local 8B vs Claude); the CI eval gate runs on a PR.
- **Phase 8:** deployed URL works; Grafana shows traffic, latency, tokens and cost; Locust results in the runbook.
- **Phase 9:** README with architecture diagram, results tables, demo GIF/video, and the docs filled in.

GETTING UNSTUCK

Each TODO's docstring tells you exactly what to build; tests show the expected behaviour. The reference solutions aren't in the repo on purpose: interviewers will ask you to explain and change this code. If you're stuck for more than 30 minutes on a function, paste your attempt and the failing test into a chat and ask for a hint, or for the reference solution for that one function.

5 • Results to measure (fill these in — they are your resume numbers)

Area	Metric	Baseline	Final
Retrieval	recall@5 / MRR (25 questions)	vector-only: ___	hybrid + rerank: ___
Assessment	scenario pass rate (20 scenarios)	local 8B: ___	production model: ___
Assessment	category accuracy · citation recall	___	___
Grounding	answers with valid citations (%)	___	___
Classifier	macro F1 (held-out)	seed only: ___	reviewed data: ___
Cost	\$ per assessment · \$ per 1k chat questions	___	with cache/fallback: ___
Latency	p95 /chat · median assessment time	___	___
Reliability	error rate under load (Locust, N users)	___	___

6 • Resume material

PROJECT LINE

EU AI Act Compliance Copilot — multi-agent RAG system that classifies AI systems under the EU AI Act and maps obligations, deadlines and gaps · FastAPI · LangGraph · PostgreSQL/pgvector · Redis/RQ · MLflow · Docker · GitHub Actions · Prometheus/Grafana · Langfuse

BULLET TEMPLATES (REPLACE THE PLACEHOLDERS WITH YOUR MEASURED NUMBERS)

- Built a **multi-agent LangGraph pipeline** (intake, rule screening, RAG-grounded classifier, obligation lookup, gap analysis, report writer, verifier) served through **FastAPI** with **Redis/RQ** background workers, **SSE** progress streaming and **human-in-the-loop** clarification.
- Implemented **hybrid retrieval** over the regulation (**pgvector HNSW** + **Postgres full-text**, reciprocal rank fusion, cross-encoder reranking) with legal-structure-aware chunking, raising **recall@5 from __ to __** on a 25-question benchmark.
- Designed an **LLMOps** layer: versioned prompts, provider abstraction with automatic fallback (**Ollama / Claude / OpenAI**), semantic caching (**__% hit rate**), per-call cost and latency tracing (**Langfuse**), and a **CI eval gate** on 20 labelled scenarios (**__% pass rate**).
- Delivered an **MLOps** lifecycle for a scikit-learn risk-area classifier used as a second-opinion model: reviewed synthetic training data, **MLflow** tracking and registry with an automated promotion gate (**macro F1 __**), and scheduled **PSI drift monitoring**.
- Shipped to production with **multi-stage Docker**, a 9-service **Docker Compose** stack and **GitHub Actions CI/CD** (lint, unit and integration tests with service containers, image build to GHCR, deploy to an EU region, smoke tests), monitored with **Prometheus/Grafana** and load-tested with **Locust** (p95 __ s at __ concurrent users).
- Built guardrails for untrusted input (prompt-injection screening, PII redaction), citation verification, and an **AI Act self-assessment and model card** for the tool itself.

2-MINUTE INTERVIEW PITCH

1. **Problem:** every EU company using AI must classify it under the AI Act; it's slow and expensive to do manually.
2. **What it does:** answers questions from the law with verified citations, and runs a multi-agent assessment that ends in a verified report with obligations, deadlines and gaps.
3. **Key design decision:** LLMs read and write, code decides what must be exact (rules, obligation lookup, verification), which makes it auditable.
4. **How I know it works:** retrieval and scenario evals, a CI gate, tracing and dashboards (quote your numbers).
5. **Production:** Docker, CI/CD, EU deployment, monitoring, drift checks.
6. **What I'd do next:** (pick one) contextual retrieval, fine-tuning a small model on reviewed assessments, a Postgres-backed LangGraph checkpointer for resumable runs.

QUESTIONS TO PREPARE (ANSWERS ARE IN THE STUDY NOTES)

- Why hybrid search, and why RRF instead of weighted score fusion? (Days 52, 57)
- How do you stop the model inventing article numbers? (Days 59, 76 + the verifier)
- Why is the obligation lookup deterministic rather than LLM-generated? (Day 67)
- Workflow or agent? Why LangGraph? What runs in parallel and why? (Days 67, 73, 74)
- How do you evaluate a multi-agent system? What's pass^k? (Day 75)
- How does the promotion gate work, and what is PSI? (Days 29, 85, S4)
- What happens when Claude is down? When Ollama is slow? (Days 47, 87)

- Walk me through a deploy and a rollback. (Days 86, 89)
 - What are the OWASP LLM Top 10 risks for this system? (Day 88)
 - How does the AI Act apply to your own tool? (S7 + the self-assessment)
-

7 • Stretch goals

- **Contextual retrieval** (LLM-written context per chunk) and measure the gain (Day 62).
- **Postgres-backed LangGraph checkpoint** so runs survive worker restarts and resume after clarification.
- **Recitals and Commission guidelines** as a second corpus with source filters.
- **Fine-tune** a small open model on reviewed assessments and compare with the prompted model (Days 79–84).
- **Multi-tenant auth** (organisations, per-tenant assessments) and rate limiting in the API.
- **MCP server** exposing `search_ai_act` and `assess_system` as tools for other agents (Day 70).
- **Kubernetes/Helm** or **Terraform** for the infrastructure.